

技术咨询报告 · TECHNICAL ADVISORY REPORT

PostgreSQL 系 AI 原生后端选型报告： 五大方案与开源组合栈全景对比

Supabase / InsForge / Nubase / ParadeDB / TimescaleDB 与无锁定组合栈的六维评估、
规模分层、应用类型方案与风险台账

2026-08-16

版本 v1.0

目录

第一章 执行摘要	3
第二章 五大方案逐卡	5
第三章 六维评分矩阵	10
第四章 规模分层技术采用分析	14
第五章 应用类型解决方案	17
第六章 SQLite Local Memory × PostgreSQL 的协同与演进	21
第七章 风险登记册与验证清单	24
附录：来源清单	27

第一章 执行摘要

1.1 一句话结论与全景判断

一句话结论：2026 年的 PostgreSQL 系后端选型没有通吃赢家——求省心与生态选 Supabase 托管版，求零锁定与私有云交付选开源自托管组合栈，求 AI 编码代理（Agent）原生工作流选 InsForge；Nubase 真实但稚嫩，当前仅作技术观察对象。

本报告以 2026-08-16 核查的事实为底座，对五个方案形态（Supabase 托管版、Supabase 自托管版、InsForge、Nubase、开源组合栈）按六个维度（开发便利性 / PG 原生度 / 可扩展性 / 模块不锁定 / 私有云可转化度 / 周边生态）逐一打分。等权合计全景如下（缩略预览，**评分方法与逐维度证据详见第三章**）：

方案	等权合计（满分 30）	一句话定位
开源组合栈	25	每层可替换、数据即物理库，唯一已落地私有云打包
InsForge	21	Agent 操作面最完整的 BaaS，3 容器自托管
Supabase 托管版	19	生态断层第一的行业基准，但锁定与计量成本真实存在
Supabase 自托管版	19	共享生态但平台功能带不走，运维自担
Nubase	14	620 stars 对 0 issues 信号异常，不宜生产

排序本身是价值观的产物：把"开发便利性"加权到 40%，Supabase 托管版反超第一；把"不锁定 + 私有云"加权到 35% 以上，组合栈断层领先。先定权重，再看排序。

需要同时指出：本报告的范围不止"中枢选型"。第六章论证了端侧（SQLite local-first 记忆）与中枢（PG 系统记录）的双层分工——"SQLite 是我的记忆，PG 是我们的事实"，并给出从单机到团队同步再到企业中枢的三阶段演进路径；记忆逻辑（时间衰减/MMR/冲突合并）可随规模逐层下沉为 PG 存储过程与 pgai 任务，API 形状全程不变。这意味着第一章的选型结论天然兼容未来的 local-first 扩展，不存在"选了 PG 底座就锁死端侧路线"的问题。

1.2 三个核心判断

判断一：Supabase 仍是基准，但正被两面夹击。它的生态优势（SDK、教程、人才、集成）在 2026 年依然断层第一，托管版开箱即用的"省心"价值对多数团队仍是正确默认。但两个方向的压力是真实的：向上，20+ 计量项（磁盘 \$0.125/GB、出口 \$0.09/GB、PITR \$100/月 add-on 等）使成本随规模线性恶化，迁出意味着重建 Auth/Storage/Realtime 整套周边栈；向下，PG17 镜像弃用 TimescaleDB 事件证明扩展决策权在平台而不在用户——"用的是真 PG"与"PG 归你管"是两回事。

判断二：2026 年的分化轴是"Agent 操作面"。BaaS 的功能面（Auth/Storage/Realtime/API）已经高度同质化，新竞争点是谁把后端变成 AI 编码代理可直接操作的对象。InsForge 的 MCP 语义层、CLI

JSON 输出、backend branching 与 diagnose 自愈是当前五方案中最完整的 agent DX（12.7k stars、日更提交）；Nubase 内置 mem0 风格 Memory 与 AI Gateway 是同一方向上的另一种押注，但其 0 issues、无 HA、v0.1.x 的稚嫩度决定了它暂时只是信号而非选项。

判断三：PG 扩展生态使"一个库干五件事"在中等规模成立。 pgvector（向量）+ pg_search 0.25.2（BM25 混合搜索）+ TimescaleDB 2.29.1（时序/连续聚合）+ pgai（自动 embedding 管道）+ DuckDB（嵌入式 OLAP）的组合，让 OLTP、全文搜索、向量检索、时序、分析五类负载在 TB 以内收敛到一个 PG 底座，省掉 ES/向量库两条同步管道。代价是两条硬约束：pg_search 的 AGPL-3.0 与 TimescaleDB 高级功能的 TSL 在企业采购中需法务书面确认；且 pg_search 实质上是自管 PG 专属（RDS 不支持、Neon 已禁用）。

1.3 选型速查表（规模 × 应用类型）

	个人 / MVP (<1万 DAU)	中小 SaaS (1万-10万)	企业级 (>10万)
CRUD 为主	Supabase Free 起步；agent 生成 CRUD 工作流选 InsForge 自托管	三轴选边：省心 Supabase Pro / agent 流 InsForge Pro / 成本确定 €22 自管栈	自管 PG + PostgREST/Hasura + Keycloak/Cerbos, BaaS 托管版退场
Search 需求	PG tsvector + GIN（中文 +zhparser），百万行内别上 ES	自管 PG + pg_search 做 BM25/混合搜索（先过 AGPL 法务关）	百亿级文档或跨地域分片才上 ES/ClickHouse, PG 保留系统记录 + CDC
Data-intensive / AI	ParadeDB 一镜像（PG+pgvector+pg_search），DuckDB 做即席分析	加 pgai 自动 embedding 管道，重分析卸载到 DuckDB	TB 级 / 亚秒聚合 SLA 命中时引入 ClickHouse, PG 不退场
金融交易数据	PG（ACID+RLS+WAL 审计）单层起步	叠 TimescaleDB hypertable + 连续聚合做行情 K 线 + DuckDB 回测	三层组合全自托管保审计链；kdb+/ClickHouse 仅在量化触发条件命中时进入

行动建议：① 先按 1.1 的权重自检明确价值取向，再查表选型；② 任何规模都不要选 Nubase 上生产，Supabase 自托管不要当中小 SaaS 主力；③ 采用 InsForge 自托管前务必显式设置 ENCRYPTION_KEY（密钥回退链 #905/#1552 未闭合）；④ 采购含 pg_search（AGPL）或 TimescaleDB 高级功能（TSL）的栈前，先过法务。

第二章 五大方案逐卡

本章是全报告的事实底座：对五个候选方案逐一建立"方案卡"——一句话定位、架构组件、关键事实（版本/许可证/stars/定价）、优势、劣势与风险、成熟度判定。所有数字均来自 2026-08-16 的核查调研并在章末「来源」列出 URL；厂商自测数据已标注，未独立复现的信息如实说明。五张方案卡之外，2.6 节给出一个"无平台锁定"的开源组合参照系，作为衡量各方案锁定程度的对照基准。

2.1 Supabase：托管基准

定位一句话：PostgreSQL 托管后端的事实标准，所有"PG 系 AI 原生后端"都在对标它。

架构组件：核心组件多年未变——Postgres（官方镜像）、Auth（GoTrue）、PostgREST、Realtime、Storage、Studio、Kong 网关、Supervisor 连接池、Logflare 分析^[1]。

关键事实：开源主体为宽松许可证；托管版定价为 Free \$0 / **Pro \$25/月/组织**（2023-09 起按组织计费，网传"\$25/项目"是过时说法），Pro 含 \$10/月 compute credit 恰好抵一台 Micro 实例；每项目独立计费的 compute 从 Micro \$10 至 16XL \$3,730；计量项包括数据库磁盘 \$0.125/GB、出口流量 \$0.09/GB、Auth MAU 超 100K 后 \$0.00325/MAU、PITR 为 \$100/月/7 天留存付费 add-on^[2]。

优势：生态最厚（SDK、教程、人才、第三方集成），托管版开箱即用，对团队最"省心"；开源组件矩阵完整，是可对标的完整功能集。

劣势/风险：① 自托管仍是"community-supported"而非官方 SLA——2025-10 官方才开 discussion #39820 收集自托管痛点；托管备份/PITR、分支、高级指标、ETL、管理 API 均为平台专属，自托管 Studio 只是功能子集^[1]。② 扩展决策随官方镜像走：**PG17 镜像已不再内置 TimescaleDB**（PG15 保留至 EOL，官方推荐 pg_partman + 原生分区），自托管需自建镜像^[3]。③ 20+ 计量项的打包计费使成本与"纯 Postgres"不可直接对比，迁出意味着重建 Auth/Storage/Realtime 周边栈^[2]。

成熟度判定：生产级基准（托管版成熟，自托管版"可用但运维自担"）。

2.2 InsForge：Agent 原生控制面的 BaaS

定位一句话：YC（P26）支持的 Apache-2.0 "agent-native" 后端平台，差异化不在 PG 层，而在给 AI 编码代理用的 MCP 语义层与 CLI 控制面。

架构组件：自托管最小栈仅 **3 个核心容器**（postgres + postgrest + insforge app），app 内含 dashboard、自研 Auth、storage、MCP 与 Deno 函数代理；数据库为 **PostgreSQL 15**（镜像 v15.13.4，含 pgvector、pg_cron），REST 走 PostgREST v12.2.12；Edge Functions 用 Deno；Realtime 用 Socket.IO 而非 PG 逻辑复制；Model Gateway 强绑定 OpenRouter^[4]。Agent 侧原语：MCP 工具（读 schema、跑迁移、部署函数等）、CLI JSON 输出、config-as-code（insforge.toml

plan/apply)、**backend branching**（整后端克隆+merge/reset）、**insforge diagnose** 诊断自修复^[5]。

关键事实：Apache-2.0；GitHub API 实测 **12,757 stars / 1,138 forks / 125 open issues**（2026-08-14，第三方旧文称 2.3k，以 API 为准），~2,780 commits，每日多次提交，最新 release v1.5.6^[6]。托管定价：Free \$0（5 万 MAU、闲置 1 周自动暂停）、Pro \$25/月（100k MAU、8GB DB、含 \$10 Compute 抵扣）、Enterprise 有 SOC2/HIPAA/SSO^[7]。

优势：面向 agent 的操作面是当前五方案中最完整的（HN 2026-05 Show HN 62 分，社区正面评价集中在"RLS 默认开启 + MCP 给 sane defaults"）；自托管极轻（3 容器）；免费层够原型^[8]。

劣势/风险：① **密钥回退链：**issue #905（已关闭）确认 ENCRYPTION_KEY 未设置时静默回退用 JWT_SECRET 加密全部 secrets——轮转 JWT_SECRET 会致 secrets 永久不可读；compose 文件当前仍有 **ENCRYPTION_KEY:-\${JWT_SECRET:-dev-secret...}** 回退链，#1552（fail-closed 加固）仍 open，自托管务必显式设置 ENCRYPTION_KEY^[9]。② PG 版本偏旧（15 而非 16/17），扩展生态保守（未见 pg_net/pgmq/TimescaleDB）。③ 官方自测 MCPMark 基准（vs Supabase 1.6x 快、1.7x 准确率）为厂商自测，**未独立复现**。④ 项目年轻（repo 2025-07 创建）、文档/社区薄、Compute 仍 private preview、Sites 依赖 Vercel^{[6][8]}。

成熟度判定：成长早期、开发高度活跃；可用于原型与中小项目，生产采用前须先堵密钥回退并跑通 fresh install。

2.3 Nubase：真实但稚嫩的开源新势力

定位一句话：2026-06-08 才开源的 "AI 原生自托管后端 + 部署层"，八大模块（DB/Auth/Storage/Assets/Functions/AI Gateway/Memory/cron）一镜像打包——真实性已核实，但稚嫩度同样真实。

架构组件：Java **Spring Boot 3.2** + Java 17 后端、Next.js Studio 前端、**database-per-project 多租户 PostgreSQL**（每项目独立物理库，非共享 schema；RoutingDataSource + 每租户 HikariCP 连接池）；**/rest/v1/*** 是 Java 重写的 PostgREST 兼容层而非独立 PostgREST；Auth 为 GoTrue 兼容（含 MFA/TOTP/OAuth）；内置一等公民 **Memory**（**mem0 兼容 API**，**pgvector 余弦 + BM25 ts_rank_cd + 实体加权混合检索**，中文 zhparser 支持）与 **AI Gateway**（OpenAI/Anthropic 兼容、每项目 **nbk_** key、成本统计）^[10]。

关键事实：GitHub **OtterMind/Nubase**：约 **620 stars / 73 forks / 106 commits / 10 contributors / 5 releases**（最新 **v0.1.4**，2026-06-16），语言 Java 76.1%，**Apache-2.0**，2026-06-08 首次开源，至 2026-08-11 仍活跃提交；有 npm 包 **nubase_cli**、Docker 镜像 **ottermind/nubase**、官方文档站与中文 README 佐证^[11]。

优势：功能叙事与代码一一对应，mem0 风格记忆层 + AI 网关是五方案中唯一"内置"的；多租户 database-per-project 是与 Supabase 自托管（单项目）的实质架构差异；Apache-2.0 + 自托管免

费。

劣势/风险（疑点如实并列）：① **620 stars 但 GitHub Issues 为 0**、仅 5 个 PR、4 watchers——社区互动与 star 数异常不匹配，star 可能主要来自营销/聚合站导流^[12]。② 第三方收录站 skillsllm.com 的安全扫描记录显示 README 曾被标记 3 处 **"secret-exfiltration" 中危提示**（instruction 疑似引导发送凭据到外部端点，状态仍为 PASSED）——未定性为恶意，但 MCP 工具安全面需读者自行评估^[12]。③ **无公司信息**：官网无公司名/地址/团队页，GitHub 组织 OtterMind 为个人型组织；中文提交信息与 WeChat OAuth 指向中文团队背景（未验证团队身份）。④ **无定价页**：nubase.ai/pricing 返回 504，README 中"AI 网关计费系统"提交暗示未来托管付费。⑤ Reddit/HN/知乎未检索到实质社区讨论。⑥ 功能缺口：无 Realtime、无备份/PITR、无 HA、无 SSO/SCIM，官方自警"暴露公网前需审查管理端点"^{[10][11]}。

成熟度判定：真实但稚嫩，不宜生产。v0.1.x + 两个月历史 + 异常社区信号，建议仅作技术观察对象；验证清单（安全审计、fresh install、备份演练）见第七章。

2.4 ParadeDB: PG 发行版，不是 BaaS

定位一句话：把 BM25 全文/混合搜索做进 PostgreSQL 的发行版——它是数据库层组件，不提供 Auth/Storage/Realtime，不能与 BaaS 直接比功能面。

架构组件：核心为 `pg_search` 扩展（BM25 索引、分面、混合搜索），发行镜像同时打包 `pgvector`；0.25.0 起 `pgvector` 成为 `pg_search` 的前置依赖（依赖其 `vector` 类型）^[13]。

关键事实：最新版 `pg_search 0.25.2`（Docker tag `paradedb/paradedb:0.25.2` 同步，文档更新于 2026-08-11）；许可证 **AGPL-3.0**（注意网络条款义务）；**pg_analytics 仓库已于 2025-03-19 归档只读**，analytics 能力并入 `pg_search` 主扩展（0.15.9 changelog 声明 2025-04-02 起弃用）；公司为 YC S23，最近一轮融资 **2025-04 Series A1 \$12M**，2026 年无新融资/收购公开证据（收购传闻未验证，按无证据处理）^{[13][14][15]}。

优势：PG 内一站式 BM25 + 向量混合搜索，省掉 Elasticsearch 同步管道；为 Postgres 15+ 提供预编译 deb/rpm/macOS 二进制（如 `postgresql-18-pg-search_0.25.2 deb`），手工安装路径清晰。

劣势/风险：① **AGPL-3.0**：网络服务场景有开源义务，商业采购需法务确认（详见第七章）。② 安装后必须 `shared_preload_libraries = 'pg_search'` 并重启，运维门槛高于普通扩展。③ 托管支持面收窄：**AWS RDS 不支持**（未列入许可扩展清单）；**Neon 已于 2026-03-19 起对新项目禁用 pg_search**，存量安装 2026 年 9 月移除，官方迁移目标是 tsvector 系的 `lakebase_text`——这是"扩展层依赖托管商决策"风险的现实案例^[16]。

成熟度判定：核心扩展生产可用（0.25.x 迭代活跃），但作为平台不是完整后端；采用形态应是"自选 PG 发行版/扩展"，而非"平台"。

2.5 TimescaleDB (Tiger Data)：时序与连续聚合老将

定位一句话：PG 时序/连续聚合扩展的老牌厂商，2025-06 公司更名 **Tiger Data**（产品名 TimescaleDB 保留，托管云更名 Tiger Cloud）。

架构组件：hypertable 自动分区、连续聚合（2.28 起支持手动增量刷新、在线 `ADD COLUMN`）、列式压缩（Hypercore，官方口径 90–95% 压缩率，厂商自测口径未独立复现）、保留策略；`timescale/timescaledb-ha` 镜像基于 Ubuntu + Patroni，内置 `pgvector`、`pgvector scale` 与 `pgai`，`shared_preload_libraries='timescaledb,vector'` 即可一镜像双能力^{[17][18]}。

关键事实：最新版 2.29.1（2026-08-04 发布）；2.29.0 起移除 PostgreSQL 15 支持，仅支持 PG 16/17/18；许可证双轨——**Apache-2.0**（核心版，可商用含转售）+ **TSL**（Timescale License，source-available 非 OSI 开源；含连续聚合、列压、保留策略等高级功能，自用免费、禁止作为托管 DBaaS 出售），法律主体已更名 "Timescale, Inc. d/b/a Tiger Data"^{[17][18]}。

优势：时序场景（行情、IoT、指标）在 PG 生态内无可替代；连续聚合做 K 线/降采样是成熟打法；`timescaledb-ha` 自带向量与 AI 扩展使其具备"时序 + AI"复合潜力。

劣势/风险：① **TSL 双轨：**高级功能不在 Apache 范围内，转售/托管场景受限，采购需逐功能核对许可证归属。② **Supabase 已在 PG17 项目上弃用 TimescaleDB**（推荐 `pg_partman` + 原生分区）——托管生态位在收缩，采用者应预期"自管 PG 上自建"成为主流路径^[3]。③ PG15 支持移除意味着存量 PG15 用户被推向升级。

成熟度判定：生产级、久经考验；但定位是"时序/分析增强层"，不是 AI 原生后端平台，需与 BaaS 层组合使用。

2.6 参照系：无平台锁定的开源组合栈

定位一句话：五个方案之外的第六种形态——用可独立替换的开源组件自行拼装 BaaS，作为衡量各方案锁定程度的对照基准；本项目已交付可运行参考实现（`ai-memory-backend/baas/`）。

架构组件（参考栈组件映射）：`paradedb/paradedb`（PostgreSQL + `pgvector` + `pg_search`）替代 Supabase Postgres；`postgrest/postgrest` 替代自动 REST API；Authentik 替代 GoTrue；Mercure（SSE）替代 Realtime；MinIO 替代 Storage；Traefik 替代 Kong 网关，统一路由 `/api`、`/auth`、`/realtime`、`/minio`^[19]。PostgREST 为可选层（可删、可换 Hasura/pREST）。配套 memweave 提供 SQLite 本地记忆层（FTS5 BM25 + 向量混合、时间衰减、MMR 重排，pytest 7/7 通过），与 PG 中枢构成第六章讨论的"端侧—中枢"协同。

关键事实：整套栈常驻内存约 4–6 GB，一台 4C8G VPS（约 €22/月，如 Hetzner CX32）可承载日活 1–10 万规模；各层可独立拆分替换（如 Neon 换自建 PG、Cloudflare R2 换 MinIO）。已提供懒猫私有云（LazyCat）LPK 打包配置与一键部署脚本（`baas/lazycat/deploy.sh`，`lzc-cli` 打包安装到盒

子)，是五方案中唯一直接面向家用/私有云场景落地的路径^{[19][20]}。验证状态：memweave 测试 7/7 通过；baas compose YAML 解析通过，**容器实机启动未验证**（开发环境无 Docker）。

优势：零平台锁定、成本透明（€22/月 vs 托管计量计费）、许可证可逐组件选择（可避开 AGPL 组件替换）、私有云/NAS 场景唯一可行解。

劣势/风险：运维自担（无托管备份/PITR、升级无 runbook、监控需自建）；组件间无统一控制台，DX 不如一体化平台；总拥有成本随团队规模上升。

成熟度判定：组件均为生产级开源项目，但"组合"本身需要工程能力背书；适合作为第二至五章评分与分层分析的锚点。

第三章 六维评分矩阵

本章把第二章的事实压缩为可比较的量化结论：先定义六个维度与评分方法（3.1），再给出 5 方案 × 6 维总表并逐维度分析（3.2），最后做权重敏感性检验（3.3）。所有打分都能在第二章找到对应事实，判词即该事实的浓缩；评分是"加权假设下的相对位置"，不是绝对质量判定。

3.1 维度定义与评分方法

六个维度（等权为默认假设）：

- ① **开发便利性**：上手速度（从 0 到第一个可用 API 的时间）、API 自动化程度（是否免写 CRUD）、调试体验（控制台、诊断工具、文档质量）。
- ② **PostgreSQL 原生度**：暴露给用户的是否是完整 PostgreSQL（直连、SQL 全能力、事务/RLS），以及扩展兼容面（pgvector/pg_search/TimescaleDB 等能否自由装卸，还是受平台镜像清单约束）。
- ③ **可扩展性**：性能扩展（实例升配、分区/时序/连接池能力）+ 功能扩展（函数、钩子、自定义服务接入）。
- ④ **模块不锁定**：每一层（DB/Auth/Storage/Realtime/API）可否独立替换；数据可否无损带走（标准 PG 物理库 vs 平台专有格式）；迁出时需要重建多少周边栈。
- ⑤ **懒猫私有云可转化度**：能否打包为 LPK 安装到家用/私有云盒子。考察三点：容器化程度与服务数量（越少越好转化）、密钥与依赖可否用 `stable_secret` / 内部域名替代、有无必须替换的外部依赖（如托管网关）。参照基准：组合栈已有完整 LPK（7 服务 + 路由 + TCP 转发，一键 `deploy.sh`）^[2]。
- ⑥ **周边生态**：GitHub stars/forks 与提交活跃度、扩展与第三方集成数量、教程与人才供给、社区讨论热度。

评分方法：1–5 分整数刻度，5 = 该维度五方案中最强或接近理论上限，1 = 该维度存在硬缺陷。每格附一句判词，判词必须能回溯到第二章的具体事实（版本号、容器数、许可证、stars、事件），不接受"感觉分"。评分对象为五个方案形态：**Supabase 托管版**、**Supabase 自托管**、**InsForge**、**Nubase**、**开源组合栈**。ParadeDB 与 TimescaleDB 是"底座组件"而非完整后端，不单独打分，但作为组合栈的数据库层与扩展层选项在维度②③分析中体现。

3.2 总表与逐维度分析

5 方案 × 6 维总表（括号内为一句判词）：

维度	Supabase 托管	Supabase 自托管	InsForge	Nubase	开源组合栈
----	-------------	--------------	----------	--------	-------

① 开发 便利 性	5（开箱即用， Studio/CLI/文档最成熟）	3（Studio 只是托管版 功能子集， 运维自担）	4（MCP/CLI/diagnose 面向 agent 最顺手，但 有密钥回退坑）	2（一镜像起得 来，但文档薄、 fresh install 未 验证）	2（无统一控制台，拼装 需工程能力）
② PG 原生 度	4（完整 PG 直连，但扩展 清单随官方镜像走）	4（同镜像， 可自建镜像 补扩展）	3（PG15 偏旧、扩展保 守、Realtime 用 Socket.IO 而非逻辑复 制）	3（database- per-project 真 PG，但 REST 层 是 Java 重写）	5（ParadeDB 发行版即 完整 PG，扩展自选）
③ 可 扩 展 性	4（compute 升到 16XL、Supervisor 池化， 功能扩展靠平台）	3（同上但升 级/备份无 runbook）	3（3 容器轻量， Compute 仍 private preview）	2（无 HA/备 份/Realtime，多 租户连接池待压 测）	4（每层独立扩，可上 TimescaleDB/Citus， 但需自建）
④ 模 块 不 锁 定	2（迁出需重建 Auth/Storage/Realtime 周边栈）	3（组件开源 但 8+ 服务耦 合，平台功 能带不走）	4（Apache-2.0 + PostgREST + 标准 PG，仅 OpenRouter 强绑定）	3（Apache- 2.0、数据在 PG 可带走， Memory/AI Gateway 内置逻 辑难替换）	5（每层可独立替换，数 据即物理库）
⑤ 懒 猫 可 转 化 度	1（托管服务，不可私有 化）	2（服务多、 密钥配置 重，无现成 LPK）	4（仅 3 容器，密钥需显 式化，OpenRouter 需 替换）	3（单镜像但 Java 栈资源占用 与内部服务未拆 分）	5（已有完整 LPK + deploy.sh，唯一已落 地）
⑥ 周 边 生 态	5（SDK/教程/人才/集成 全面第一）	4（共享同一 生态，自托 管议题在社 区跟进中）	3（12.7k stars、日更提 交，但社区与文档仍薄）	1（620 stars、0 issues、无实质 社区讨论）	4（组件各自生态成熟， 组合本身无统一社区）
等 权 合 计	19	19	21	14	25

维度① 开发便利性——Supabase 托管版胜。5 分无任何争议：Free 层开箱即用，SDK、教程、人才供给是事实标准。InsForge 拿 4 分靠的是对 AI 编码代理的操作面——MCP 工具、CLI JSON 输出、`insforge diagnose`、backend branching 是当前五方案中最完整的 agent DX；扣分项是 #905 密钥回退这类"第一次踩到才知道"的坑。Supabase 自托管掉到 3：Studio 只是功能子集，备份/PITR/分支都带不走。Nubase 与组合栈并列末位但原因相反：前者是"一体化但稚嫩"（v0.1.4、fresh install 未验证），后者是"成熟但需自己拼装"。

维度② PG 原生度——组合栈胜。 组合栈直接跑 ParadeDB 发行版 (PG15+、pgvector + pg_search 0.25.2)，扩展装卸自由，5 分。Supabase 两版均 4：给的是完整 PG，但 PG17 镜像弃用 TimescaleDB 事件说明扩展决策在平台而不在用户；自托管可自建镜像故不更低。InsForge 3 分：PG15 偏旧、未见 pg_net/pgmq/TimescaleDB，Realtime 用 Socket.IO 绕开了 PG 逻辑复制。Nubase 3 分：底层是 database-per-project 的真 PG (含 pgvector + zhparser)，但 `/rest/v1/*` 是 Java 重写而非 PostgREST 本体，兼容性长期风险自担。

维度③ 可扩展性——托管版与组合栈并列。 Supabase 托管的 compute 阶梯 (Micro \$10 → 16XL \$3,730) 加 Supavisor 池化是最省事的性能扩展路径，给 4 而非 5 是因为功能扩展依赖平台路线。组合栈同 4：性能上可在数据库层换 TimescaleDB (hypertable/连续聚合/列压) 或加 Citus，功能上每层独立演进，代价是全部自建。Nubase 仅 2：官方自警"暴露公网前需审查管理端点"，无 HA、无备份，database-per-project 的每租户 HikariCP 池在大规模下未经验证。

维度④ 模块不锁定——组合栈满分，托管版垫底。 组合栈每层有明名替换路径 (Hasura 换 PostgREST、R2 换 MinIO、Neon 换自建 PG)，数据即 PG 物理库，5 分。InsForge 4 分：Apache-2.0、3 容器、REST 走标准 PostgREST，数据可直接 `pg_dump` 带走，扣分在 Model Gateway 强绑定 OpenRouter。Supabase 托管版 2 分的判词来自第二章：迁出意味着重建 Auth/Storage/Realtime 整套周边栈，且 20+ 计量项使成本与"纯 Postgres"不可直接对比——这正是锁定定义本身。

维度⑤ 懒猫私有云可转化度——组合栈唯一 5 分。 参照 `baas/lazycat/`：LPK 三件套 (package.yml / lzc-manifest.yml / lzc-build.yml) + deploy.sh 已覆盖 7 服务、路由与 TCP 转发，密钥由懒猫 `stable_secret` 自动生成持久化，网关用懒猫自带 HTTPS——这是"已转化"而非"可转化"^[21]。InsForge 4 分是除组合栈外最可行的候选：仅 3 个核心容器 (postgres + postgrest + app)，服务数少意味着 manifest 改写量最小；两个待办是显式设置 ENCRYPTION_KEY (堵 #905 回退链) 和把 OpenRouter 换成可自托管的模型网关。Nubase 3 分：单镜像看似最简，但 Java Spring Boot 栈的资源占用、内部多模块未拆分为独立服务，且无任何打包先例。Supabase 自托管 2 分：8+ 服务 (Kong/GoTrue/Realtime/Storage/Studio…) + 大量密钥配置，无现成 LPK，工作量最大；托管版 1 分，按定义不可私有化。

维度⑥ 周边生态——Supabase 断层第一。 stars、SDK 覆盖、第三方集成、人才市场四项全面领先，5 分无争议；自托管版共享该生态记 4。InsForge 3 分：12,757 stars / 1,138 forks / 每日多次提交是真实活跃度，但项目 2025-07 才建、文档与社区沉淀薄。组合栈 4 分的逻辑：PostgREST、Authentik、MinIO、ParadeDB 各自都是成熟生态，只是"组合"这件事没有统一社区。Nubase 1 分且判词必须写重：620 stars 对 0 issues、5 PR、Reddit/HN/知乎零实质讨论——第二章已判定该信号异常，生态维度上它不是一个"有社区的项目"。

3.3 敏感性与误用警告

等权假设下总分为：组合栈 25 > InsForge 21 > Supabase 托管 19 = Supabase 自托管 19 > Nubase 14。这个排序本身是"不锁定与私有云各占六分之一权重"这一价值观的产物，换权重即翻转：

- 若"开发便利性"加权至 40%（其余五维均分剩余 60%）：Supabase 托管版以约 3.9 的加权分反超至第一（组合栈约 3.6、InsForge 约 3.5）。结论翻转为"初创/MVP 直接上托管，别为理论上的可迁移性付工程税"。这也是市场上 Supabase 仍是基准的现实原因——多数团队的贴现率就是很高。
- 若"模块不锁定"加权至 40%：组合栈加权分升至约 4.6，与第二名 InsForge（约 3.9）拉开断层，Supabase 托管版跌至倒数第二（约 2.9）。结论翻转为"凡涉及数据主权、合规审查、私有云交付的场景，一体化平台全部让位"。
- 翻转的临界点：维度①权重约 >30% 时托管版胜出；维度④+⑤ 合计权重约 >35% 时组合栈胜出且不可撼动。Nubase 在任何合理权重下都不进入前三——它的低分集中在稚嫩的硬事实（0 issues、无 HA、未验证），不是权重能救的。

误用警告：① 本表是 2026-08-16 时点的快照，InsForge（日更提交）与 Nubase（v0.1.x）的分数半衰期可能只有数月；② 维度⑤的 5 分依赖本项目自建的 LPK 参照实现，换其他私有云目标（群晖、K8s homelab）时容器数与密钥管理的结论方向不变、分数需重估；③ 合计分不应被引用为"某方案比某方案好 20%"这类线性陈述——维度间不可通约，正确用法是先定权重再看排序。

第四章 规模分层技术采用分析

本章沿用白皮书 2.4 节的 Tier 1/2/3 分层框架，并用 2026-08-16 核查的最新事实刷新候选名单（如 PocketBase 继续稳居 Tier 1、ParadeDB 进入 Tier 2 数据库位、Citus/Keycloak/Cerbos 进入 Tier 3），逐层回答同一个问题：**在这个规模下，选什么、备选什么、明确不要选什么**。结论先行：日活 1 万以下用 Supabase Free 或单机组合栈起步，1 万–10 万区间是三条路线分岔的真正战场，10 万以上唯一
的正确答案是把 PostgreSQL 收回自管并引入企业级身份与授权栈。

4.1 Tier 1：个人 / MVP（日活 < 1 万）

结论先行：推荐 Supabase Free 起步，次选 InsForge 自托管（3 容器），不要在这一层自建 K8s 或引入任何按席位计费的企业组件。

候选有四个：

- **Supabase Free**：\$0/月，开箱即得 Postgres + Auth + Storage + Realtime 全家桶，生态教程与人才储备最厚，原型期"省心"价值最高 [22]。
- **InsForge 自托管**：Apache-2.0，最小栈仅 **3 个核心容器**（postgres + postgrest + insforge app），RLS 默认开启，MCP/CLI 控制面让 AI 编码代理直接读 schema、跑迁移、部署函数——如果 MVP 本身就是"agent 生成 CRUD"的工作流，这是当前最顺手的操作面 [4]。托管 Free 层亦可 \$0 原型（5 万 MAU 额度，闲置 1 周自动暂停）[7]。
- **PocketBase**：白皮书 Tier 1 的原始推荐，2026 年判断维持不变——单二进制、SQLite 内嵌、内置 Auth/Realtime/Storage，`scp` 即部署，月成本约 \$5 的 VPS 即可承载，零云依赖、一个文件夹带走 [23]。
- **单机组合栈**：本项目 baas 参考栈（ParadeDB + PostgREST + Authentik + MinIO + Mercure + Traefik）常驻内存约 4–6 GB，一台 4C8G VPS 可跑，但对个人 MVP 而言组件数偏多，适合"从第一天就要求零锁定"的场景而非追求速度的 MVP [24]。

不要选：任何企业级组件（Keycloak 的运维复杂度、Cerbos 的策略工程）在 <1 万 DAU 阶段纯属过度设计；也不要选 Nubase——项目真实但仅 620 stars、v0.1.x、2026-06 才开源，无 Realtime/备份/HA/SSO，稚嫩度与 MVP 的"快速试错"诉求同样不匹配 [25]。

4.2 Tier 2：中小 SaaS（日活 1 万–10 万）

结论先行：本层无普适最优解，按"成本 / 控制力 / 运维负担"三轴选边——求省心选 Supabase Pro，求 agent 工作流选 InsForge Pro，求成本确定性与零锁定选自管组合栈。

三轴	Supabase Pro	InsForge Pro	自管组合栈 (€22 VPS)
----	--------------	--------------	-----------------

名义成本	\$25/月/组织（含 \$10 compute credit，恰抵一台 Micro） ^[2]	\$25/月（100k MAU、8GB DB、含 \$10 Compute 抵扣） ^[7]	€22/月 4C8G VPS（如 Hetzner CX32） ^[24]
隐性成本	见下方计量项分析	PG15 偏旧、扩展生态保守，Compute 仍 private preview ^[4]	备份/PITR/监控/升级全部自担，TCO 随团队规模上升
控制力	低（托管黑盒，扩展决策随官方镜像走）	中（Apache-2.0 可随时转自托管）	高（每层可独立替换）
运维负担	最低	低（3 容器）	最高（无统一控制台、无托管备份）

Supabase 计量项的隐性成本必须单独说清。\$25/组织只是入场价：数据库磁盘 \$0.125/GB、文件存储 \$0.0213/GB、出口流量 \$0.09/GB（cached \$0.03）、Auth MAU 超 100K 后 \$0.00325/MAU、PITR 为 \$100/月/7 天留存的付费 add-on（Pro 不含）^[2]。一个 50GB 数据库 + 200GB/月出口 + 需要 PITR 的项目，月度账单将显著高于 \$25——而 20+ 计量项的打包计费使成本与“纯 Postgres”不可直接对比，这是迁出时才发现的真实锁定^[2]。与之对照，€22/月的组合栈成本曲线是平的：容量上限即硬件上限，不存在“用得越多单价项越多”的惊吓，但代价是 PITR、监控、升级 runbook 全部自建。InsForge 的 \$25 介于两者之间，且其自托管路径（3 容器 + compose）使“先用托管、超标后转自托管”成为三者中迁移成本最低的选项——前提是先堵密钥回退：务必显式设置 ENCRYPTION_KEY，当前 compose 仍存在回退链（#905 已确认静默回退、#1552 加固仍 open），轮转 JWT_SECRET 会导致 secrets 永久不可读^[9]。

不要选：Supabase 自托管作为本层主力——官方定位为 community-supported 而非 SLA 产品，托管版的备份/PITR、分支、管理 API 均为平台专属，自托管 Studio 只是功能子集；且 PG17 镜像已弃用 TimescaleDB，自托管需自建镜像^[26]。同样不建议在本层就上 K8s——Docker Compose 足以承载 4C8G 单机 1-10 万日活^[24]。

4.3 Tier 3：企业级（日活 > 10万）

结论先行：这一层 BaaS 平台整体退场，正确形态是“自管 PG + 企业级身份/授权 + 专业 IaC”；同时 AGPL 与 TSL 许可证从纸面条款变成真实的采购约束。

推荐形态（沿用白皮书 Tier 3 框架并以 2026 事实刷新）：

- **数据库：**自管 PostgreSQL（或 RDS 等托管）+ **Citus** 分片做水平扩展；时序/行情数据叠 **TimescaleDB 2.29.1**（hypertable/连续聚合/列压，注意 2.29.0 起移除 PG15 支持）^[27]；搜索叠 **ParadeDB pg_search 0.25.2**（BM25 + 向量混合）^[28]。
- **身份与授权：****Keycloak**（Red Hat 背书，最全协议支持、LDAP/AD 联邦）替代 GoTrue 系；**Cerbos** 做字段级/资源级授权引擎——RLS 之外补齐复杂权限审计面^[23]。
- **部署：**Kubernetes + Terraform，监控/备份/PITR 按内部 SLO 自建；多集群 MinIO 或 Cloudflare R2 承担全球分布存储^[23]。

合规审计视角：企业采购的核心问题是可审计性与责任边界。BaaS 托管版（含 Supabase Enterprise）把审计外包给厂商 SOC2 报告；自管路线的审计对象变为自身 IaC 与 runbook。两条路都可行，但混合（核心数据自管、周边托管）在企业级通常带来最差的审计拼接成本，建议整层选边。

许可证在这一层成为硬约束：① **ParadeDB pg_search 为 AGPL-3.0**，网络服务场景触发开源义务，且 Neon 已 2026-03-19 起对新项目禁用、AWS RDS 不支持——"扩展层依赖托管商决策"的风险在企业级必须书面排除，商业采购需法务确认^[28]。② **TimescaleDB 双轨：**Apache-2.0 核心版之外的高级功能（连续聚合、列压、保留策略）属 TSL（source-available、非 OSI），自用免费但禁止作为托管 DBaaS 转售——若企业产品形态涉及对外提供数据库服务，需逐功能核对许可证归属^[27]。③ 对照之下，InsForge 与 Nubase 的 Apache-2.0 在许可证维度最干净，但前者未达企业成熟度、后者明确不宜生产^[25]。

不要选：任何 single-tenant BaaS 托管版直接扛 >10万 DAU 核心流量——计量成本（egress/MAU/磁盘）随规模线性恶化且无容量上限议价权；也不要选 PocketBase（单二进制架构到不了这一层）和 Nubase（稚嫩度问题见 4.1）。

4.4 三层速查表与升级触发条件

层	推荐	次选	明确不要选
Tier 1 (<1 万 DAU)	Supabase Free	InsForge 自托管（3 容器）/ PocketBase	企业组件（Keycloak/Cerbos）、Nubase、自建 K8s
Tier 2 (1 万-10万)	按三轴选边：省心=Supabase Pro / agent 流=InsForge Pro / 成本确定=€22 自管栈	三者互为次选	Supabase 自托管当主力、过早 K8s
Tier 3 (>10 万)	自管 PG + Citus + Keycloak/Cerbos + K8s/Terraform	BaaS Enterprise 合约（审计外包路线）	BaaS 标准托管版扛核心流量、PocketBase、Nubase

升级触发条件（出现这些信号就该往下一层迁）：

- **T1→T2：**Supabase Free 额度触顶（MAU/磁盘连续两周 >80%）；出现第一个付费客户需要 PITR 与备份 SLA；团队从 1 人扩到 3 人以上、需要迁移审查与分支。
- **T2→T3：**Supabase 月度计量账单（磁盘+egress+MAU 超额）超过自管成本的 2 倍；合规方（客户/监管）首次要求 SOC2 自证或数据驻留；单库写入成为瓶颈需 Citus 分片；RLS 已无法表达权限模型、需 Cerbos 级授权引擎；AGPL/TSL 组件进入对外售卖路径。
- **反向信号：**若自管栈的运维投入超过一名工程师 30% 工时，应考虑反向迁回托管——分层是双向的。

第五章 应用类型解决方案

第四章按"规模"分层，本章换一条轴：按**应用类型**给出"最小可行栈 + 升级路径"。四类典型负载——CRUD、Search、Data-intensive、金融交易——对 PG 生态的调用方式截然不同，同一套栈在四类负载下的优劣会反转。每节末尾给出升级触发条件，5.5 汇总成表。所有版本号、许可证与定价均为 2026-08-16 核查事实，出处见章末「来源」。

5.1 CRUD 为主：管理后台与业务工具

判断：CRUD 类应用（管理后台、内部工具、表单系统）的瓶颈不在数据库，而在"API 的边际生产成本"——谁把"建表到可调接口"的距离压到最短，谁就赢。

最小可行栈：PostgreSQL + **PostgREST** (REST) 或 **Hasura** (GraphQL + 实时订阅)。两者都做到"建表即 API"：PostgREST 通过 `PGRST_DB_SCHEMA` 暴露 schema，RLS 直接成为 API 的安全边界；Hasura 额外提供 GraphQL 联邦与 console^[29]。认证接 Authentik/Keycloak 签发的 JWT，全栈一台 4C8G VPS（约 €22/月）即可承载中小团队全部内部系统^[30]。

升级路径一：Agent 生成 CRUD。InsForge 在此有真实而非营销的价值：其 MCP 语义层向编码代理暴露 `get-table-schema`、`run-raw-sql` (strict mode)、`bulk-upsert` 等工具，并支持 agent 直接执行迁移前滚——CRUD 从"人写接口"变为"agent 读 schema 后生成表+RLS+API"^[5]。HN 社区实测评价"RLS 默认开启 + MCP 给 sane defaults 是巨大胜利"（2026-05，62 分）；官方 MCPMark 基准（vs Supabase 1.6x 快、1.7x 准确率）为**厂商自测，未独立复现**。代价是 PG 锁在 15、扩展生态保守，且自托管务必显式设置 `ENCRYPTION_KEY` 以规避密钥回退链（issue #905/#1552）。

升级路径二：多库路由。当需要把同一套自动 API 铺到多个存量库时，**pREST**（Go 实现的 PostgREST 精神续作）是更轻的路由选项：单二进制、低内存、可按实例挂载不同库，适合网关化部署^[29]。Hasura 则以多数据源联邦覆盖同一需求，但更重。

不要选：为纯 CRUD 后台引入 Realtime/Edge Functions 全家桶——80% 的内部工具用不到，徒增运维面。

5.2 Search 需求：三级阶梯

判断：PG 内搜索的天花板在 2026 年已被显著抬高，多数中小场景不再需要 Elasticsearch；但"什么时候 PG 系扛不住"必须有明确边界。

阶梯一：内置 tsvector/tsquery。零成本起步，`ts_rank_cd` 排序 + GIN 索引即可支撑百万行级的关键词检索；中文场景加 `zhparser`。这是 Neon 官方为 `pg_search` 用户指定的降级迁移目标（`lakebase_text`，基于 tsvector）的底层能力^[31]。

阶梯二：pg_search 0.25.2 (ParadeDB)。当需求升级为 BM25 相关性、分面聚合、模糊纠错、向量混合搜索时，pg_search 把 Tantivy 引擎做进 PG：最新版 **0.25.2**，0.25.0 起 pgvector 成为前置依赖，向量+BM25 一体^[31]。两个采用前提必须写进采购单：① **AGPL-3.0** 网络条款义务，商业 SaaS 需法务确认；② 安装后必须配置 `shared_preload_libraries = 'pg_search'` 并重启，且 **AWS RDS 不支持、Neon 已对新项目禁用**（存量 2026 年 9 月移除）——它实质上是“自管 PG 专属”方案^[31]。

```
-- pg_search 最小示例：BM25 索引 + 混合检索
CREATE INDEX idx_docs_bm25 ON docs
  USING bm25 (id, title, body) WITH (key_field='id');
SELECT id, paradedb.score(id) AS bm25_score
FROM docs WHERE docs @@@ 'title:数据库 OR body:后端'
ORDER BY bm25_score DESC LIMIT 20;
```

阶梯三：何时必须上 ES/ClickHouse。边界条件有三：① **文档规模到百亿级**——PG 单库即便分区也难承载该量级倒排索引的写入放大；② **跨地域分片与多活**——ES 的分片路由与跨集群复制是 PG 扩展不提供的；③ **搜索流量与 OLTP 强隔离的合规/SLA 要求**。此时正确形态是“PG 为系统记录 + 变更数据捕获（CDC）同步到 ES/ClickHouse”，而不是替换 PG^[32]。

5.3 Data-intensive：分析、报表与 AI 管道

判断：2026 年的 PG 扩展生态使“一个库干五件事”在中等规模（TB 以内）成立；超过边界后，ClickHouse/DuckDB 以互补而非替代的方式接入。

最小可行栈：ParadeDB 发行版（PG + pgvector + pg_search 一镜像）承载业务数据、全文与向量混合检索，省掉 ES + 向量库两条同步管道^[31]。

升级路径一：嵌入式 OLAP。DuckDB 是分析层的最佳互补：`pip` 级集成、进程内零运维，可直接以 `postgres_scanner` 或 `parquet` 直读方式对 PG/对象存储做即席聚合，把重分析查询从 OLTP 库卸载到笔记本或任务容器：

```
-- DuckDB 内直接扫 PG 做报表，不占生产库 CPU
INSTALL postgres; LOAD postgres;
ATTACH 'dbname=app host=pg.internal' AS pg (TYPE POSTGRES);
SELECT date_trunc('day', created_at) d, count(*), sum(amount)
FROM pg.orders GROUP BY 1 ORDER BY 1;
```

升级路径二：自动 embedding 管道。pgai（Timescale/Tiger Data 出品，`timescaledb-ha` 镜像已内置）在 PG 内定义 vectorizer：行插入即自动调用 embedding 模型、写回向量列，把“业务表 → 向量索引”的 ETL 从应用代码里消掉^[33]。与 pgvector 组合后，AI 记忆与业务数据同库，审计与备份策略统一。

边界：① **分析数据量到 TB 级以上、QPS 要求亚秒聚合**——上 ClickHouse 做列存分析层，PG 保留系统记录；② **实时写入吞吐超单库**（>10 万行/秒持续）——Kafka + ClickHouse 管道。ParadeDB 的

pg_analytics 仓库已归档只读 (2025-03-19)，其能力并入 pg_search，说明 PG 内 OLAP 路线收敛于"搜索增强"而非"替代列存数仓" [31]。

5.4 金融交易数据：三层组合

核心论点：金融场景的正确答案不是"选一个最快的库"，而是用 **PG 系三层组合（OLTP 系统记录 + TimescaleDB 行情时序 + DuckDB 回测）** 同时满足 **ACID 正确性、时序查询效率与监管可审计性**——kdb+/ClickHouse 只在明确的规模触发条件下才进入。

第一层：OLTP 系统记录 = PostgreSQL。 订单、成交、持仓、资金流水必须落在具备完整 ACID、行级安全 (RLS) 与触发器/WAL 审计链的库上——这是监管视角下"系统记录" (system of record) 的唯一合格形态：数据不出域（自管或专有云）、RLS 做账户级隔离、WAL + PITR 提供完整可重放的审计轨迹。Supabase/InsForge 这类 BaaS 的 RLS 默认开启实践 (HN 实测正面评价) 证明该模式已被工程化 [5]。

第二层：行情与 K 线 = TimescaleDB hypertable + 连续聚合。 Tick/分钟线写入 hypertable 自动分区，连续聚合物化 5 分/小时/日 K 线并增量刷新 (2.28 起支持手动增量刷新、轻锁)，列式压缩（官方口径 90-95%，厂商自测）+ 保留策略控制存储成本；当前版本 **2.29.1** (2026-08-04, 已移除 PG15 支持) [32][17]：

```
-- 连续聚合生成 5 分钟 K 线 (TSL 许可证功能)
CREATE MATERIALIZED VIEW kline_5m WITH (timescaledb.continuous) AS
SELECT time_bucket('5 minutes', ts) AS bucket, symbol,
       first(price, ts) AS open, max(price) AS high,
       min(price) AS low, last(price, ts) AS close, sum(volume) AS vol
FROM ticks GROUP BY bucket, symbol;
```

注意许可证与兼容性含义：连续聚合属 **TSL** (source-available, 自用免费、禁止作为托管 DBaaS 转售) [17]；且 **Supabase PG17 镜像已弃用 TimescaleDB**——若行情层跑在托管 PG 上，应预期迁移到自管 PG 或官方推荐的 pg_partman + 原生分区路线，这一托管生态位收缩是采购决策的现实变量 [3]。

第三层：回测与量化分析 = DuckDB。 回测是典型"读多、可重跑、无并发写入"负载：DuckDB 直接扫 TimescaleDB 导出的 parquet 或经 postgres_scanner 连库拉数，进程内列存执行，策略迭代不需要动生产库（示例见 5.3）。数据同时满足"研究自由度"与"生产隔离"两个互相冲突的要求。

kdb+/ClickHouse 的触发条件：① **Tick 级数据存量超 TB 且要求毫秒级全历史扫描**（高频做市/全市场回放）→ kdb+；② **多资产全市场行情写入持续 >10 万行/秒、查询以全库聚合为主** → ClickHouse 作为行情分析层，PG 仍保留交易记录 [32]。未触发前，引入它们意味着额外团队、许可证成本（kdb+ 为商业闭源）与同步管道，属于为不存在的问题付利息。

监管友好性小结：三层全部为可自托管软件，数据不出域；交易层 WAL 审计链 + 时序层保留策略 + 分析层只读快照，构成"不可变记录—派生视图—研究副本"的清晰证据链，比"一切进黑盒商业引擎"在合规答辩时更有利。

5.5 汇总：四类应用 × 最小栈/升级栈

应用类型	最小可行栈	升级栈（触发条件）	明确不要选
CRUD 为主	PG + PostgREST（或 Hasura）+ Authentik JWT	InsForge（agent 生成 CRUD/迁移）； pREST/Hasura 联邦（多库路由）	全量 BaaS 全家桶
Search 需求	PG tsvector + GIN（中文 +zhparser）	pg_search 0.25.2（BM25/混合搜索；自 管 PG 专属）；ES/ClickHouse + CDC（百 亿文档/跨地域分片）	百万行级就 上 ES 集群
Data-intensive	ParadeDB 一镜像 （PG+pgvector+pg_search）	DuckDB 嵌入 OLAP；pgai 自动 embedding；ClickHouse（TB 级/亚秒聚 合 SLA）	自建 ES+向 量库双管道
金融交易数据	PG（OLTP+RLS+WAL 审计）+ TimescaleDB（行情/K 线）+ DuckDB（回 测）	kdb+（TB 级 tick 毫秒回放）； ClickHouse（>10 万行/秒全市场行情）	交易记录进 非 ACID 存 储

四类的共同主线：默认从"PG + 扩展"出发，只在量化触发条件命中时引入专用引擎——PG 系在 2026 年能覆盖的负载面比多数团队的直觉更宽，而每一次外引专用引擎都应以可测量的边界为依据，而非以惯性为依据。

第六章 SQLite Local Memory × PostgreSQL 的协同与演进

前五章比较的是"中枢侧"的方案选型；本章回答一个更前瞻的问题：当团队已经有（或将有）PG 底座，又想用 SQLite 做 local-first 的 AI 记忆，两者如何分工、同步、共同演进？结论先行：**这不是二选一，而是"端侧真相源 + 中枢系统记录"的双层架构**——SQLite 赢在它天然就是端侧的唯一选项（零部署、离线、隐私），PG 赢在它天然就是中枢的唯一选项（多租户、审计、共享）。本报告项目内已交付的 memweave 参考实现（FTS5+向量混合搜索、时间衰减、MMR、WAL 多 Agent 并发、Markdown 真相源，pytest 7/7 通过）将作为全章的工程参照^[34]。

6.1 分工模型：端侧真相源 × 中枢系统记录

分工的第一性原理是"数据的物理归属"：随人/随设备走的数据放 SQLite，随组织/随系统走的数据放 PG。memweave 的设计把这条边界落实得很干净——`memory.db` 单文件 + `memory/*.md` Markdown 真相源可以 git 管理、整体拷走，这是 PG 永远做不到的部署形态；而跨用户共享、合规审计、服务多租户，则是 SQLite 永远无法承担的角色。

数据类别	放哪边	为什么
个人偏好、对话上下文、Agent 工作记忆	SQLite (memweave)	隐私敏感、离线可写、随设备迁移、零成本
Markdown 记忆真相源	SQLite 侧文件系统	可 git diff、可人工审阅，PG 里反而不可读
用户账号、权限、计费、审计日志	PG	多租户强一致、RLS、合规留痕
跨团队共享的知识沉淀、已脱敏记忆聚合	PG	需要共享检索与权限治理
高频写、低价值的临时观察	SQLite (异步汇聚)	避免端侧写入打爆中枢连接池

一句话判词：**SQLite 是"我的记忆"，PG 是"我们的事实"**；混淆两者（把个人记忆直接写中枢，或把共享事实留在端侧）是最常见的架构错误。

6.2 同步与互操作路径（按可实现性排序）

路径一：应用层批量同步（今天就能做，推荐起点）。用 Python 脚本定期把 memweave 的实体 UPSERT 进 PG，简单、可控、易调试：

```
import sqlite3, psycopg
src = sqlite3.connect("memweave-data/memory.db")
dst = psycopg.connect("postgresql://app@hub/central")
rows = src.execute(
    "SELECT session_id,key,value,confidence,updated_at FROM memory_entities").fetchall()
with dst.cursor() as cur:
    cur.executemany("""
        INSERT INTO memory_hub(session_id,key,value,confidence,updated_at)
        VALUES (%s,%s,%s,%s,%s)
        ON CONFLICT (session_id,key) DO UPDATE
        SET value=EXCLUDED.value, updated_at=EXCLUDED.updated_at
        WHERE memory_hub.updated_at < EXCLUDED.updated_at""", rows)
dst.commit()
```

要点：以 `(session_id, key)` 为幂等键，用 `updated_at` 做 last-write-wins 裁决——与 memweave 内置的冲突检测语义对齐，同步天然幂等，可放心跑在 cron 里。

路径二：DuckDB 作为查询胶水（本周就能做，分析场景首选）。 DuckDB 可同时 ATTACH SQLite 文件和 PG 库，一条 SQL 完成跨库联合查询，无需落地任何管道^[35]：

```
ATTACH 'memweave-data/memory.db' AS mem (TYPE SQLITE);
ATTACH 'postgresql://app@hub/central' AS hub (TYPE POSTGRES);
SELECT e.key, e.value, u.plan
FROM mem.memory_entities e
JOIN hub.users u ON u.session_id = e.session_id
WHERE e.confidence > 0.8;
```

要点：这是“读侧互操作”的最优解——分析师/Agent 需要联合查询时即席执行，不需要预先同步；配合第五章的 DuckDB 嵌入式 OLAP 定位，它在这里扮演的是“联邦查询胶水”而非存储层。

路径三：PG FDW / 逻辑复制（长期方向，谨慎评估）。理论上可用 `sqlite_fdw` 让 PG 直接挂载端侧库、或以逻辑复制订阅端侧变更流；但现实中端侧设备不常驻、网络不稳定，FDW 的推拉模型并不适合“离线优先”拓扑。此路径更适合“端侧常驻在线”的特殊形态（如门店网关设备），一般团队建议止步于路径一/二。

6.3 扩展层演进：记忆逻辑下沉进 PG

memweave 的三块核心记忆逻辑——时间衰减（ $0.95^{\text{days}} \times (1+0.1 \cdot \text{access_count})$ ）、MMR 多样性重排、冲突合并——目前是约百行级 Python。白皮书已明确判断：SQLite 方案牺牲 mem0 的开箱即用，换取数据主权，记忆逻辑层“需 100-200 行 Python 自行实现”^[36]。关键在于，这 100-200 行代码是**可迁移资产而非沉没成本**：

- **时间衰减**本质是排序键上的一个标量函数，写成 PG 存储过程不过十余行 PL/pgSQL，或直接内联进 `ORDER BY`；配合 pgvector 的 `ORDER BY embedding <=> $q` 即可在同一查询里完成“向量召回 ×

时间衰减”。

- **MMR** 是"贪心选下一个与已选集合最不相似的结果"，可用存储过程循环 + pgvector 距离算子实现，或作为 pgai 的后处理任务。
- **冲突合并** 的 `(session_id,key)` UPSERT 语义与 PG 的 `ON CONFLICT` 一一对应（见 6.2 代码），甚至因为 PG 有条件更新（`WHERE ... updated_at < EXCLUDED.updated_at`）而表达得更精确。
- **向量化回灌**：端侧 SQLite 可以只存文本，embedding 由 PG 侧的 pgai 任务统一生成（避免端侧跑模型），检索时混合 pg_search 的 BM25——这正是 ParadeDB"一个库做混合搜索"（2.4 节）的自然延伸^[37]。

概念映射上，**SQLite-Sync 的 CRDT ↔ PG 的逻辑复制**是对偶关系：前者解决"无中心的副本最终一致"，后者解决"有中心的变更分发"。一个系统里两者可以共存——端侧之间走 CRDT，端侧到中枢走 6.2 的 UPSERT 汇聚，中枢内部走逻辑复制。全章最有想象力的一点在此：**"记忆逻辑下沉"不是重写，而是同一套衰减/重排语义从 Python 层向数据库层的逐行平移，规模每上一个台阶就下沉一层，API 形状保持不变**——这正是 PG 扩展生态（第三章"可扩展性"维度）在记忆场景的直接兑现。

6.4 三阶段演进图

从单机到企业中枢的演进不需要一次到位，每一阶段的栈都是完整可用的，迁移由明确的信号触发：

	阶段一：单机 local-first	阶段二：团队同步	阶段三：企业中枢
栈形态	memweave 单文件（memory.db + Markdown 真相源），纯标准库即可运行	memweave 端侧 × N + 6.2 路径一 UPSERT 汇聚进 PG + PostgREST 暴露只读 API	PG + pgvector + pgai 承接记忆逻辑（存储过程/pgai 任务），SQLite 退居端侧缓存与离线缓冲
记忆逻辑位置	Python（100-200 行）	仍在端侧；PG 侧只做汇聚与授权检索	下沉进 PG（PL/pgSQL + pgai 任务），端侧仅剩写入缓冲
迁移触发信号	——	出现第二个需要共享记忆的用户/Agent；合规要求留存；单机备份成为心病	记忆条目 > 百万级或 QPS 超单机；需要跨租户治理/审计；端侧 embedding 质量成为瓶颈
数据迁移成本	——	低：UPSERT 脚本即迁移工具，Markdown 真相源不动	中：schema 一次映射 + 逻辑重写为存储过程（语义不变）；Markdown 真相源仍可保留为导出格式

三点补充判断：① 阶段二是最容易被跳过也最值得停留的阶段——多数团队的"共享记忆"需求用 UPSERT + PostgREST 就能满足，不必过早引入 CRDT 或 FDW。② 阶段三不等于抛弃 SQLite：离线写入、端侧隐私这两个属性永远属于端侧，SQLite 从"真相源"降级为"缓存+缓冲"是健康的架构退位而非失败。③ 三阶段的 API 形状（write_memory / search）全程不变，这正是 6.3"逐层下沉"策略的回报——演进改的是数据位置，不是调用方代码。

第七章 风险登记册与验证清单

本章把第二至第六章分散的风险事实收敛为两张可直接使用的表：7.1 是按方案登记的风险台账（含严重度与缓解措施），7.2 是任何方案采用前都应跑完一遍的验证清单。7.3 以一段话收束全报告。

7.1 风险登记册

严重度口径：**高** = 可导致数据丢失/不可读、法务风险或生产不可用，采用前必须处置；**中** = 有明确缓解措施、不处置则随规模放大；**低** = 需知晓但可接受。每条风险均可回溯到第二章的事实来源。

#	方案	风险	严重度	缓解措施
R1	InsForge	密钥回退链：ENCRYPTION_KEY 未设置时静默回退用 JWT_SECRET 加密全部 secrets（#905 已关闭确认行为，#1552 fail-closed 加固仍 open）；轮转 JWT_SECRET 将致 secrets 永久不可读	高	部署时 显式设置独立 ENCRYPTION_KEY 并纳入密钥管理（私有云场景用 <code>stable_secret</code> ）；升级前核对 compose 是否仍含 <code>:-\${JWT_SECRET:-...}</code> 回退链；跟踪 #1552 合并状态
R2	InsForge	项目年轻（repo 2025-07 创建）、文档/社区薄、Compute 仍 private preview、MCPMark 基准为厂商自测未独立复现	中	限定用于原型与中小项目；关键路径自测（见 7.2）；不把厂商基准数字写入容量规划
R3	Nubase	稚嫩且社区信号异常：v0.1.4、开源两个月、620 stars 但 GitHub 0 issues /5 PR、Reddit/HN/知乎零实质讨论——star 疑似主要来自营销/聚合站导流	高	不作生产采用，仅作技术观察对象；如需试用，隔离网络环境部署，不放真实数据
R4	Nubase	第三方收录站 skillsllm 安全扫描记录显示 README 曾被标记 3 处 "secret-exfiltration" 中危提示 （instruction 疑似引导发送凭据到外部端点，状态 PASSED，未定性为恶意） ^[38]	中	接入任何 MCP/agent 工具链前人工审计其 prompts 与网络出口；凭据最小授权
R5	Nubase	无公司信息（官网无公司名/团队页）、无定价页（/pricing 返回 504）、功能缺口（无 Realtime/备份/PITR/HA/SSO），官方自警"暴露公网前需审查管理端点"	高	按"无供应商背书"对待：自建备份与监控，管理端点加网关鉴权后方可暴露
R6	Supabase 托管	计量成本不可预测 ：20+ 计量项（磁盘 \$0.125/GB、出口 \$0.09/GB、MAU 超 100K 后 \$0.00325/MAU、PITR \$100/月 add-on），成本与"纯 Postgres"不可直接对比 ^[39]	中	上线前用真实流量模型做月度账单推演；设用量告警；定期与"4C8G VPS €22/月组合栈"基准对账
R7	Supabase 托管	托管锁定 ：备份/PITR、分支、高级指标、ETL、管理 API 为平台专属；迁出需重建 Auth/Storage/Realtime 整套周边栈	中	数据层坚持标准 SQL 与 pg_dump 可导出形态；对 GoTrue/Storage/Realtime 的使用面做接口封装；每年做一次退出成本测算（见 7.2）

R8	Supabase (托管/自托管)	扩展决策随官方镜像走： PG17 镜像已弃用 TimescaleDB (推荐 pg_partman + 原生分区)	低	依赖扩展前先确认目标镜像清单；自托管可自建镜像补回，但计入运维成本
R9	ParadeDB pg_search	AGPL-3.0 ：网络服务 (SaaS) 场景触发开源义务，且 AWS RDS 不支持安装 ^[40]	高 (商用场景)	商用采购前法务确认 AGPL 义务或购买商业授权；对锁定敏感的组合栈可用 tsvector/lakebase_bm25 系方案替代
R10	TimescaleDB	TSL 双轨 ：连续聚合、列压、保留策略等高级功能在 Timescale License 下，自用免费但 禁止作为托管 DBaaS 转售 ；2.29.0 起移除 PG15 支持	中	逐功能核对所用特性归属 Apache 还是 TSL (可用 oss 标签镜像规避)；有转售/托管业务模型的须法务确认；存量 PG15 用户安排升级
R11	开源组合栈	运维自担 ：无托管备份/PITR、PG 大版本升级无 runbook、监控需自建；组合本身无统一社区	中	自建 pgBackRest 备份 + 恢复演练制度化；升级前在副本环境演练；把运维工时计入 TCO 对比
R12	任意托管 PG	托管扩展政策风险 ：Neon 2026-03-19 起对新项目禁用 pg_search、存量 2026 年 9 月移除并导向自家 lakebase_text —— 托管商可单方面改变扩展供给 ^[41]	中	核心查询能力避免绑定单一托管商独有扩展；优先选用可自安装的扩展；把"扩展清单变更"列入供应商监控项

7.2 采用前验证清单

以下六项按执行顺序排列，任何方案（含托管版）在进入生产前都应打勾；Nubase 等稚嫩项目须全部通过且结果留档，成熟托管服务至少完成 3–6 项：

- ☐ **Fresh install 跑通**：在干净机器/干净账号从零部署到"第一个真实 API 调用成功"，记录耗时与卡点。预期坑位：InsForge 的密钥配置、Nubase 的未验证安装路径、组合栈的容器实机启动（本项目 compose 仅通过 YAML 解析验证）。
- ☐ **密钥显式化**：枚举全部 secret（数据库口令、JWT_SECRET、ENCRYPTION_KEY、网关 token），确认每一处都有独立、持久、可轮转的赋值，无默认值回退链；私有云场景用 **stable_secret** 类机制托管。
- ☐ **备份恢复演练**：不是"备份配置好了"，而是实际做一次**恢复**——从备份还原到独立实例并校验数据行数与关键业务查询。Supabase 托管用户须确认 PITR 是否为付费 add-on；自托管与组合栈用 pgBackRest 建立基线。
- ☐ **压测基线**：对核心读写路径压出 p95 延迟与吞吐基线（CRUD、搜索、实时通道各一），并与方案宣称数字（尤其厂商自测基准）对照；Nubase 的每租户 HikariCP 池、InsForge 的 Socket.IO Realtime 是各自未经公开验证的重点项。
- ☐ **许可证法务确认**：逐项确认 pg_search（AGPL-3.0 网络条款）、TimescaleDB（TSL 转售限制）、各平台主体许可证（Apache-2.0/MIT）在自身业务模型下的义务；商用 SaaS 场景对 AGPL 组件给出"购买商业授权 / 替换组件 / 自研规避"三选一的明文结论。

- □ **退出成本测算**：演练一次 `pg_dump` 全量导出与异机恢复，估算重建周边栈（Auth/Storage/Realtime/搜索）的人日与双跑期成本，写入采购决策文档——退出成本是锁定风险的量化形态，应每年复测。

7.3 总结论

回到全报告最初的问题——“哪个 PG 系 AI 原生后端最好”——本报告的答案是：**没有最好的 BaaS，只有与你的规模、应用类型、锁定容忍度相匹配的栈**。Supabase 以生态和省心守住基准位，InsForge 在 agent 操作面上开辟了新的竞争轴，Nubase 提醒我们“AI 原生”叙事下仍需核实稚嫩与异常信号，ParadeDB 与 TimescaleDB 证明数据库层的能力分化仍在加速——而贯穿五章评分、风险台账与验证清单的同一条结论是：**PostgreSQL 生态真正的护城河不是任何单一平台，而是可组合性**——每一层可独立替换、数据始终是物理库、退出路径永远存在，这才是对供应商决策风险（密钥回退、许可证变更、扩展弃用）最根本的对冲。

附录：来源清单

（以下来源均于 2026-08-16 访问；按正文首次引用顺序编号）

- [1] [1] <https://queryglow.com/blog/supabase-self-hosted>
（自托管 community-supported、组件清单、平台专属功能）
- [2] [2] <https://selfhost.dev/blog/supabase-pricing-explained/> ； <https://omidsaffari.com/blog/supabase-pricing>
（定价模型，2026-08 对照官网核验）
- [3] [3] <https://basekick.net/blog/timescaledb-alternatives-2026> ； <https://www.supascale.app/blog/timescaledb-for-selfhosted-supabase-time-series-data-guide> （Supabase PG17 弃用 TimescaleDB）
- [4] [4] <https://docs.insforge.dev/core-concepts/database/overview.md> ； <https://raw.githubusercontent.com/InsForge/InsForge/main/deploy/docker-compose/docker-compose.yml> （3 容器栈、PG15）
- [5] [5] <https://docs.insforge.dev/mcp-setup.md> ； <https://docs.insforge.dev/agent-native/overview.md> ； HN Algolia story 48181342 （2026-05，62 分）； GitHub InsForge issues #905/#1552 （密钥回退）；官方 MCPMark 为厂商自测
- [6] [6] <https://github.com/InsForge/InsForge> 及 GitHub API /repos/InsForge/InsForge （stars/forks/commits，2026-08-14）
- [7] [7] <https://insforge.dev/pricing>
- [8] [8] HN Algolia story 48181342 （2026-05，62 分）； <https://agent-finder.co/reviews/insforge> （独立实测 7/10）； <https://www.developersdigest.tech/blog/agent-native-backends-insforge>
- [9] [9] GitHub InsForge issues #905 / #1552 / #890: <https://github.com/InsForge/InsForge/issues> ； compose 回退链 <https://raw.githubusercontent.com/InsForge/InsForge/main/deploy/docker-compose/docker-compose.yml>
- [10] [10] <https://nubase.ai/features> ； <https://nubase.ai/docs> 、 [/docs/concepts](https://nubase.ai/docs/concepts) 、 [/docs/getting-started](https://nubase.ai/docs/getting-started)
- [11] [11] <https://github.com/OtterMind/Nubase> 及仓库 README、docs/architecture.md
- [12] [12] <https://skillsllm.com/skill/nubase> （安全扫描记录）； <https://ossinsight.io/analyze/OtterMind/Nubase> ； GitHub Issues/PR 列表 （0 issues / 5 PR）
- [13] [13] <https://docs.paradedb.com/deploy/upgrading> ； <https://docs.paradedb.com/deploy/self-hosted/extension> ； <https://github.com/paradedb/paradedb/releases>
- [14] [14] https://github.com/paradedb/pg_analytics （归档声明）； <https://docs.paradedb.com/changelog/0.15.9>
- [15] [15] <https://pitchbook.com/profiles/company/534519-73> （Series A1 \$12M）
- [16] [16] https://neon.com/docs/extensions/pg_search ； <https://neon.com/docs/extensions/migrate-pg-search-to-lakebase-text> ； https://pgxn.org/dist/pg_search/0.18.5/pg_search/README.html （RDS 不支持声明）
- [17] [17] <https://github.com/timescale/timescaledb/releases> （2.29.1 / 2026-08-04 / 移除 PG15 支持）
- [18] [18] <https://www.tigerdata.com/legal/licenses> ； <https://docs.tigerdata.com/about/latest/timescaledb-editions/> ； <https://github.com/timescale/timescaledb-docker-ha>
- [19] [19] 本项目仓库 /mnt/agents/output/ai-memory-backend/baas/README.md （组件映射、成本说明、验证状态）
- [20] [20] 本项目仓库 /mnt/agents/output/ai-memory-backend/baas/lazycat/ （LPK 打包与 deploy.sh）

- [21] [21] 本项目仓库 `/mnt/agents/output/ai-memory-backend/baas/lazycat/README.md`（LPK 打包规则、stable_secret、7 服务 manifest、deploy.sh）；懒猫官方转换规则 <https://developer.lazycat.cloud/app-example-porting.html>
- [22] [22] <https://supabase.com/pricing>（Free/Pro 档位，2026-08 核验）
- [23] [23] `/mnt/agents/upload/AI_Memory_and_Backend_Alternatives_Whitepaper.md` §2.4（Tier 1/2/3 框架，本报告以 2026 事实刷新）
- [24] [24] `/mnt/agents/output/ai-memory-backend/baas/README.md`（€22/月 4C8G、4–6GB 常驻内存、1–10 万日活容量判断）
- [25] [25] <https://github.com/OtterMind/Nubase>；<https://nubase.ai/docs>（620 stars/v0.1.4/功能缺口）
- [26] [26] <https://queryglow.com/blog/supabase-self-hosted>；<https://basekick.net/blog/timescaledb-alternatives-2026>（自托管 community-supported、PG17 弃用 TimescaleDB）
- [27] [27] <https://www.tigerdata.com/legal/licenses>；<https://github.com/timescale/timescaledb/releases>（TSL 双轨、2.29.1、PG15 支持移除）
- [28] [28] <https://github.com/paradedb/paradedb/releases>（0.25.2/AGPL-3.0）；https://neon.com/docs/extensions/pg_search（Neon 禁用）；https://pgxn.org/dist/pg_search/0.18.5/pg_search/README.html（RDS 不支持）
- [29] [29] <https://github.com/PostgREST/postgrest>；<https://github.com/prest/prest>；<https://hasura.io>（自动 API 层组件）
- [30] [30] 本项目仓库 `/mnt/agents/output/ai-memory-backend/baas/README.md`（组合栈成本与组件映射）
- [31] [31] <https://docs.paradedb.com/deploy/upgrading>；<https://docs.paradedb.com/deploy/self-hosted/extension>；https://github.com/paradedb/pg_analytics（归档声明）；https://neon.com/docs/extensions/pg_search；<https://neon.com/docs/extensions/migrate-pg-search-to-lakebase-text>；https://pgxn.org/dist/pg_search/0.18.5/pg_search/README.html（RDS 不支持声明）；AGPL-3.0 许可证
- [32] [32] <https://www.tigerdata.com/docs/get-started/news/new>；<https://www.tigerdata.com/legal/licenses>（TSL 双轨：连续聚合/列压/保留策略归属）
- [33] [33] <https://github.com/timescale/timescaledb-docker-ha>（timescaledb-ha 内置 pgvector/pgvectorscale/pgai）
- [34] [34] memweave 参考实现说明（FTS5+向量混合搜索、时间衰减公式、MMR $\lambda=0.7$ 、WAL 多 Agent、Markdown 真相源、pytest 7/7），`/mnt/agents/output/ai-memory-backend/memweave/README.md`（访问日期 2026-08-16）。
- [35] [35] DuckDB SQLite/Postgres 扩展 ATTACH 用法，DuckDB 官方文档 duckdb.org（访问日期 2026-08-16）。
- [36] [36] 白皮书《AI Memory and Backend Alternatives Whitepaper》第一部分（1.1–1.7：SQLite 记忆层架构、mem0 对比表、CRDT 同步、“记忆逻辑层 100-200 行 Python”结论），`/mnt/agents/upload/AI_Memory_and_Backend_Alternatives_Whitepaper.md`（访问日期 2026-08-16）。
- [37] [37] 本报告第二章 2.4（ParadeDB pg_search 混合搜索）、2.6（组合栈参照与 memweave 衔接），`/mnt/agents/output/pg-backend-comparison_sec02.md`（访问日期 2026-08-16）。
- [38] [38] <https://skillsllm.com/skill/nubase>（安全扫描记录）；<https://github.com/OtterMind/Nubase>（0 issues / 620 stars）；<https://ossinsight.io/analyze/OtterMind/Nubase>
- [39] [39] <https://selfhost.dev/blog/supabase-pricing-explained/>；<https://queryglow.com/blog/supabase-self-hosted>（计量项、平台专属功能）
- [40] [40] <https://docs.paradedb.com/deploy/self-hosted/extension>；https://pgxn.org/dist/pg_search/0.18.5/pg_search/README.html（AGPL-3.0、RDS 不支持）
- [41] [41] https://neon.com/docs/extensions/pg_search；<https://neon.com/docs/extensions/migrate-pg-search-to-lakebase-text>（Neon 弃用 pg_search 事件）